Automata Explorer - Detailed Proposal

Meilir Parry
Student ID: 201582632

October 2024

Contents

1 Statement of Ethical Compliance

This project falls into the data and participant category A2 and I confirm that I will follow the University of Liverpool's ethical guidance.

2 Project Description

Automata theory is a fundamental topic in computer science that studies abstract machines and their computational capabilities. It is a core component of the computer science curriculum, providing a theoretical foundation for understanding the limits of computation. This project aims to develop a web-based interactive platform for teaching automata theory. Educators will be provided the facility to create and assign problems to their students as well as the tools to monitor and analyse students' answers. Students will solve the problems by drawing state diagrams or inputting regular expressions, with the system providing real-time feedback on the correctness of their answers. In addition to this, an AI language model will be implemented to generate additional problems for students on request, allowing for further practice if desired.

3 Aims & Requirements

3.1 Aims

- **Develop an interactive learning environment**: To create an interactive and optimal tool for students that simplifies the learning of automata theory through visualisation and practice. Efficient and intuitive tools will be provided for students to solve problems and receive immediate feedback.
- **Facilitate teacher insight into student learning**: Enable teachers to track student progress and identify common difficulties or misconceptions, thus, enabling teachers to adapt their teaching to improve students' knowledge.
- **Enhance learning with AI**: Integrate an AI model that generates further problems for the student to practice and enhance learning.

3.2 Requirements

3.2.1 Essential

- **Automata Algorithms**: Algorithms to validate DFA, NFA, ε-NFA, regular expressions, and pushdown automata. Algorithms for the conversion of different automata and for the simulation of automata for given inputs or regular expressions.

- **Student Interface**: An intuitive and responsive web interface where students can solve problems generated by teachers or AI by visually drawing state diagrams or inputting regular expressions.
- **Teacher Interface**: Dashboard for teachers to view students' progression, add custom problems, or modify existing ones.
- **Real-time solution validation**: Students' state diagrams will be converted to machine-readable format for validation against the correct solution. Problems formulated by teachers will also be validated and confirmed.
- **AI Problem Generation**: Generate problems with AI, enabling more material for students to practice and learn.

3.2.2 Desirable

- **Turing Machines**: Extend the project to include Turing machines.
- **Collaborative Features**: Allow students to collaborate with each other to solve problems.
- **Accessibility Features**: Develop an interface that is accessible to users with disabilities.
- **Advanced AI Features**: Generate tailored problems to students based on their progression analytics.
- **Real-time Hints**: Display hints to students if progression on solving a problem is slow or repeatedly incorrect.

4 Key Literature & Background Reading

The core topic of this project is automata theory, which focuses on the definition and properties of mathematical models of computation. There are different kinds of models of computation, each capable of recognising classes of languages. As stated in the description, I will concentrate on finite automata (deterministic, nondeterministic with and without ε transitions), regular expressions, and pushdown automata. Most of the theory is from the book "Introduction to the Theory of Computation" by Michael Sipser [1]. This book provides a comprehensive introduction to automata theory, formal languages, and computability theory, making it an essential resource for this project. The book covers the theory of automata in great detail, including the definitions of finite automata, regular expressions, and pushdown automata.

Finite automata are models for computers with limited memory, formally defined as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is a finite set called states, Σ is a finite set called the alphabet, which indicates the permitted input symbols, δ is called the transition function, defined as $\delta : Q \times \Sigma \to Q$, q_0 is the starting

state where $q_o \in Q$, and F is a set of accepting states where $F \subseteq Q$ [1]. The transition function δ defines the rules for moving from one state to another, based on the input. $\delta(x, 1) = y$ indicates that if the automaton is in state x and reads input symbol 2, it will move to state y. The automaton accepts an input string if it ends in an accepting state after reading the entire input. This kind of automaton is called a deterministic finite automaton (DFA), where for each state and input symbol, there is exactly one transition to another state. In contrast, a nondeterministic finite automaton (NFA) can have multiple transitions for a state and input symbol. Moreover, an NFA can take as input the empty string ε, which allows it to move to another state without reading any input. Formally, the transition function, δ, for an NFA is defined as follows: $\delta : Q \times (\Sigma \times \{\varepsilon\}) \to P(Q)$, where $P(Q)$ is the power set of Q [1].

Abstractly, a finite automaton can be represented as a directed graph where the nodes are the states and the edges are the transitions between states. The edges are labelled with the input symbol that causes the transition. These are called state diagrams, which provide a visual representation of the automaton. For example, the DFA in Figure 1 accepts strings that end in 1.

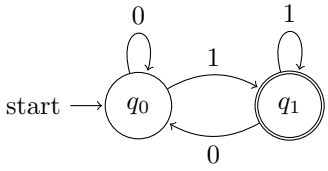

Figure 1: A state diagram of a DFA accepting strings ending in 1

Using the formal definition above, this DFA can be defined as $(\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$, where δ is defined as:

$$\delta(q_0, 0) = q_0$$
$$\delta(q_0, 1) = q_1$$
$$\delta(q_1, 0) = q_0$$
$$\delta(q_1, 1) = q_1$$

The computation performed by a finite automaton can be formally defined as follows: Let M be a finite automaton, defined as a 5-tuple from the above definition, and let $w = w_1, w_2, \ldots, w_n$ be a string where for each w_i, $w_i \in \Sigma$.M is said to accept w if a sequence of states $r_0, r_1, \ldots, r_n \in Q$ exists with three conditions: $r_0 = q_0$; $\delta(r_i, w_{i+1}) = r_{i+1}$, for $i = 0, \ldots, n-1$; $r_n \in F$. The language accepted by M, denoted $L(M)$, is the set of all strings that M accepts [1].

Regular expressions are another model for defining languages, which can be converted to finite automata and vice versa. R is a regular expression if R is: a for some a in the alphabet Σ; ε, which denotes the empty string; $\emptyset$;

$(R_1 \cup R_2)$, where R_1 and R_2 are regular expressions; $(R_1 \cdot R_2)$, where R_1 and R_2 are regular expressions; (R_1^*), where R_1 is a regular expression [1]. For example, the regular expression $(0 \cup 1)^*1$ denotes the set of all strings that end in 1, which is equivalent to the DFA in Figure 1.

Pushdown automata are slightly more complex models of computation that include a stack for memory. They are capable of recognising context-free languages, which are more powerful than regular languages. A pushdown automaton is formally defined as a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, Γ is the stack alphabet, δ is the transition function, q_0 is the starting state, and F is the set of accepting states. The transition function δ is defined as $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \to P(Q \times (\Gamma \cup \{\varepsilon\}))$, where P is the power set of a set [1].

For implementing the algorithms to convert an NFA to a DFA, construct an NFA from a regular expression, and to convert an ε-NFA to an NFA, the book "Automata Theory: An Algorithmic Approach" by Javier Esparza and Michael Blondin [2] will be used. This book provides a practical approach to automata theory, focusing on the algorithms and their implementation, and therefore, will be a valuable resource for understanding the algorithms required for this project. Converting an NFA to a DFA is commonly done using the *powerset construction* method. The algorithm for this method is shown in Algorithm 1. Algorithm 1 maintains a hash table W, which stores the sets of states that have not been processed, then iterates over the states in W, adding the states to the DFA, marking them as accepting states if they contain an accepting state from the NFA, and adding transitions to the DFA based on the transitions in the NFA.

Algorithm 1 Powerset Construction [2]

Input: NFA $A = (Q, \Sigma, \delta, q_0, F)$
Output: DFA $B = (\mathcal{Q}, \Sigma, \delta', q_0', F')$
$\mathcal{Q}, \delta', F' \leftarrow \emptyset$
$q_0' \leftarrow q_0$
$W \leftarrow \{q_0\}$
while $W \neq \emptyset$ **do**
    pick Q' from W
    add Q' to $\mathcal{Q}$
    if $Q' \cap F \neq \emptyset$ **then**
        add Q' to F'
    end if
    for all $a \in \Sigma$ **do**
        $Q'' \leftarrow \bigcup_{q \in Q'} \delta(q, a)$
        if $Q'' \notin \mathcal{Q}$ **then**
            add Q'' to W
        end if
        add (Q', a, Q'') to δ'
    end for
end while

Algorithm 2 shows the algorithm for converting an ε-NFA to an NFA. Similar to Algorithm 1, it maintains a hash table W to store the set of states to iterative over. A transition such as $q_1 = \delta(q_0, a)$ where $q_0, q_1 \in Q$ and $a \in \Sigma$ is represented as a three tuple (q_0, a, q_1). a denotes an element of Σ, and α, β are elements of $\Sigma \cup \{\varepsilon\}$. If the ε-NFA has transitions (q, α, q') and (q', β, q''), such that $\alpha = \varepsilon$ or $\beta = \varepsilon$, then the transitions can be combined into a single transition $(q, \alpha\beta, q'')$, removing the ε transitions [2].

Algorithm 2 Conversion of ε-NFA to NFA [2]

Input: ε-NFA $A = (Q, \Sigma, \delta, q_0, F)$
Output: DFA $B = (Q', \Sigma, \delta', q_0', F')$
$q_0' \leftarrow q_0$
$Q' \leftarrow \{q_0\}$
$\delta' \leftarrow \emptyset$
$F' \leftarrow F \cap \{q_0\}$
$\delta'' \leftarrow \emptyset$
$W \leftarrow \{(q, \alpha, q') \in \delta \mid q = q_0\}$
while $W \neq \emptyset$ **do**
    pick (q_1, α, q_2) from W
    if $\alpha \neq \varepsilon$ **then**
        add q_2 to Q'
        add (q_1, α, q_2) to δ'
        if $q_2 \in F$ **then**
            add q_2 to F'
        end if
        for all $q_3 \in \delta(q_2, \varepsilon)$ **do**
            if $(q_1, \alpha, q_3) \notin \delta'$ **then**
                add (q_1, α, q_3) to W
            end if
        end for
        for all $a \in \Sigma, q_3 \in \delta(q_2, a)$ **do**
            if $(q_2, a, q_3) \notin \delta'$ **then**
                add (q_2, a, q_3) to W
            end if
        end for
    else
        add (q_1, α, q_2) to δ''
        if $q_2 \in F$ **then**
            add q_2 to F'
        end if
        for all $\beta \in \Sigma \cup \{\varepsilon\}, q_3 \in \delta(q_2, \beta)$ **do**
            if $(q_1, \beta, q_3) \notin \delta' \cup \delta''$ **then**
                add (q_1, β, q_3) to W
            end if
        end for
    end if
end while

The Thompson's construction algorithm is used to convert a regular expression into an equivalent ε-NFA. This method outlines a few rules that can be applied to a regular expression, R, which are described in the book "Introduction to Algorithms" [1] as:

1. When $R = a$ for some $a \in \Sigma$, then the equivalent NFA is defined as

NFA $= (\{q_1, q_2\}, \Sigma, \delta, q_1, \{q_2\})$ where $\delta(q_1, a) = \{q_2\}$ and $\delta(r, b) = \emptyset$ for $r \neq q_1$ or $b \neq a$.

2. When $R = \varepsilon$, the equivalent NFA is defined as NFA $= (\{q_1\}, \Sigma, \delta, q_1, \{q_1\})$ where $\delta(r, b) = \emptyset$.

3. When $R = \emptyset$, the equivalent NFA is defined as NFA $= (\{q\}, \Sigma, \delta, q, \emptyset)$ where $\delta(r, b) = \emptyset$.

4. When $R = R_1 \cup R_2$, the equivalent NFA is constructed as follows: let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be the NFAs of R_1 and R_2 respectively. Constructing $N = N_1 \cup N_2 = (Q, \Sigma, \delta, q_0, F)$:

   - $Q = \{q_0\} \cup Q_1 \cup Q_2$
   - $F = F_1 \cup F_2$
   - for any $q \in Q$ and any $a \in (\Sigma \cup \{\varepsilon\})$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ and } a = \varepsilon \\ \emptyset & q = q_0 \text{ and } a \neq \varepsilon \end{cases}$$

5. When $R_1 \cdot R_2$, the equivalent NFA is defined as follows: let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be the NFAs of R_1 and R_2 respectively. Constructing $N = N_1 \cdot N_2 = (Q, \Sigma, \delta, q_1, F_2)$:

   - $Q = Q_1 \cup Q_2$
   -

$$\delta = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \varepsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ and } a = \varepsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

6. $R = R_1^*$ is constructed as follows: let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ be the NFA for R_1. Constructing $N = (Q, \sigma, \delta, q_0, F)$ for R_1^*:

   - $Q = \{q_0\} \cup Q_1$

   - $F = \{q_0\} \cup F_1$
   - for any $q \in Q$ and any $a \in (\Sigma \cup \{\varepsilon\})$

$$\delta = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \varepsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ and } a = \varepsilon \\ \{q_1\} & q = q_0 \text{ and } a = \varepsilon \\ \emptyset & q = q_0 \text{ and } a \neq \varepsilon \end{cases}$$

State diagrams will be represented as a collection of adjacency lists in the C++ code. This idea was inspired by the book "Introduction to Algorithms" by Thomas H. Cormen [3], which provides a detailed explanation of graph representations and algorithms. The adjacency list representation is efficient for storing the state diagrams and allows for easy traversal and manipulation of the graphs.

MongoDB, which is a NoSQL database, provides their own documentation [4] on how to configure databases and integrate them with Node.js. Moreover, mongoose, a Node.js library for MongoDB, provides a detailed guide on how to interact with MongoDB using Node.js [5].

The Llama API documentation [6] provides a small guide on how to interact with the API, including the endpoints and the required parameters. This will be used to implement the AI model for problem generation, which will be a key feature of the project.

5 Development & Implementation Summary

The project will primarily developed using two programming languages: C++ and JavaScript. The core algorithms will be developed with C++, while JavaScript, along with HTML/CSS, will be used to implement the web interface. WebAssembly [7] and Emscripten [8] will then be used to compile C++ code to run in the web browser, providing almost native performance. The back-end will be developed using Node.js and Express.js [9], with a MongoDB database for storing user logins, problems, and the students' answers. For the AI aspect of the project, I will make use of Meta's large language model Llama using the service provided by [6]. This service is relatively cheap when compared to other services such as OpenAI's ChatGPT.

The development of the project can be broken down into four core components, sequentially developed as follows:

1. Develop the core algorithms for the automata theory in C++:
2. Develop the back-end using Node.js and Express.js, and set up and integrate the MongoDB database.
3. Develop the web interface using JavaScript, HTML, and CSS.
4. Implement the AI model for problem generation.

Each component will undergo thorough testing alongside its development, reducing the potential issues that may occur when developing the succeeding component.

The first component focuses on designing and the implementation of the various algorithms required to simulate and validate both teachers' and students' answers. These algorithms will be capable of:

- Simulating DFAs on an input formatted as adjacency lists.

- Conversion of an NFA and ε-NFA to an equivalent DFA.
- Constructing a DFA/NFA from a regular expression.
- Validating whether a language is regular using the pumping lemma theorem.
- Simulating pushdown automata, including stack operations for context-free grammar recognition.

These algorithms will be developed using the C++ programming language because of the language's performance and efficiency, which will contribute to ensuring an optimal user experience. During this phase, the algorithms will be tested via a command line interface to confirm their functionality before proceeding to the next component.

The next component is the development of the back-end side of the web application. Developed using Node.js and Express.js, this component will be responsible for handling the communication between the front-end interface and the database. This component will be developed in parallel with the database setup and integration to ensure a seamless transition between the two components. Node.js and Express.js provide an efficient and scalable solution for the back-end, and as I have experience with these technologies, I am confident in their implementation. The configuring of the MongoDB database will be done using the cloud service MongoDB Atlas [4], which provides an intuitive interface for managing databases and collections. The database will store information such as user logins, problems, and students' answers, allowing for easy retrieval and modification of data. Mongoose, a Node.js library for MongoDB, will be used to interact with the database, providing a simple and efficient way to perform CRUD operations [5].

Following this will be the implementation of the web interface, developed using JavaScript, HTML, and CSS. This component will provide an interactive and responsive interface for both the teachers and the students, although their interfaces will differ slightly in functionality and design. To aid with the development of the front-end, I'll make use of the Bootstrap framework [10] which will improve the responsiveness and aesthetics of the web application.

Finally, the last component of the project will be the implementation of the AI model for problem generation. This will be developed using the Llama API, which is an API wrapper for Meta's large language model, Llama. This will facilitate the generation of problems for students to practice, enhancing their learning experience, and will be implemented after the completion of the web interface. By sending a JSON object containing the prompt through the API, the model will generate a problem based on the prompt, which will then be sent back to the web application, validated by the algorithms, and presented to the student.

Throughout the development process, I will make use of Git and GitHub [11] for version control. This will allow me to track changes, revert to previous versions if necessary, and, by branching, allow me to work on different components simultaneously without affecting the main codebase.

6 Data Sources

There will be two sources of data for this project: the data generated by Llama API and the user data stored in the MongoDB database. Data from Llama API will first be obtained by making a JSON request to the API, which will then return a JSON object. This JSON object will then be handled by the back-end of the web application, and validated to ensure the data is suitable.

User data will include:

- username
- password
- class group
- problems created by the teacher
- students' answers

The user data will be stored in the MongoDB database, which will be accessed and modified using the Mongoose library. The data will be stored securely, with passwords hashed using bcrypt [12] to ensure confidentiality and anonymity of personal information. The data will only be accessible to the teacher and the student who created the data, ensuring that the data remains confidential and secure.

7 Testing & Evaluation

Testing will be conducted throughout the development of the project to ensure each component functions correctly and meets the requirements outlined in the proposal. I will be using a combination of unit testing, integration testing, and system testing. Unit testing will focus on testing individual functions and algorithms to ensure they produce the expected output, such using the CLI to test the C++ algorithms. Integration testing will ensure that the components work together as expected, for example, testing the communication between the front-end and back-end. System testing will involve testing the entire system as a whole, ensuring that the web application functions correctly and meets the requirements outlined in the proposal. This includes testing the web interface, the back-end, and the AI model together to ensure they work seamlessly.

Evaluating the project will involve comparing the final product to the original requirements stated above. This will involve assessing the usability of the web interface, the performance and accuracy of the algorithms, and the effectiveness of the AI model in generating problems. Additionally, feedback will be obtained from people who have briefly tested the platform, such as students and hopefully teachers, to identify any issues and to examine whether the platform meets the requirements.

8 Project Ethics & Human Participants

The project will involve human participants during the evaluation phase, in which students and teachers will be asked to test the platform and provide feedback. The evaluation will involve students solving problems on the platform and providing feedback on the usability of the interface, the effectiveness of the AI model, and the overall learning experience. Teachers will be demonstrated the platform and asked whether it meets their requirements for teaching automata theory. They will be asked to complete a small survey, which will ask questions about the platform's usability and effectiveness. Participants will provide consent before participating in the evaluation, and they'll be made aware of what data will be collected and how it will be used. All personal identities will be removed from the data set and replaced with an anonymous identifier to ensure confidentiality. The data set itself will be stored securely in a password-protected document, accessible only to me, and potentially, my project supervisor. A participant can withdraw during the evaluation upon request, without any explanation needed. If the request is made before the data is anonymised, the data will be deleted.

9 BcS Project Criteria

An ability to apply practical and analytical skills gained during the degree programme.	The project applies many modules learnt through the years, including the applications of algorithms, computational theory, database development, and software engineering principles.
Innovation and/or creativity.	The project aims to innovate the way automata theory is taught by providing an interactive and engaging platform for students to learn. The integration of an AI model for problem generation is a creative solution to enhance the learning experience.
Synthesis of information, ideas and practices to provide a quality solution together with an evaluation of that solution.	The project will synthesise information from various sources, including textbooks on automata theory and AI models, to provide a quality solution. The project will be evaluated based on the requirements outlined in the proposal.
That your project meets a real need in a wider context.	Automata theory is a fundamental topic in computer science, and this project aims to deliver an educational platform that can enhance the learning experience for students, and provide teachers with valuable insights into student progress.
An ability to self-manage a significant piece of work.	Breaking down the project into smaller components and functions will ensure the project remains manageable. Regular testing and evaluation will be conducted to ensure the project is on track.
Critical self-evaluation of the process.	Regular testing and evaluation will be conducted throughout the project, and feedback will be obtained from users to identify any issues and areas for improvement.

10 UI/UX Mockup

Figure 2 shows the UI/UX for the web interface, where user is presented with a problem and is asked to draw a state diagram to represent the automaton that accepts the language described by the problem. This is the perspective of the student. Figure 3 shows the teacher's perspective, where the teacher can create a problem which is verified by the system before being assigned to students.

These diagrams were created using the online tool Figma [13].

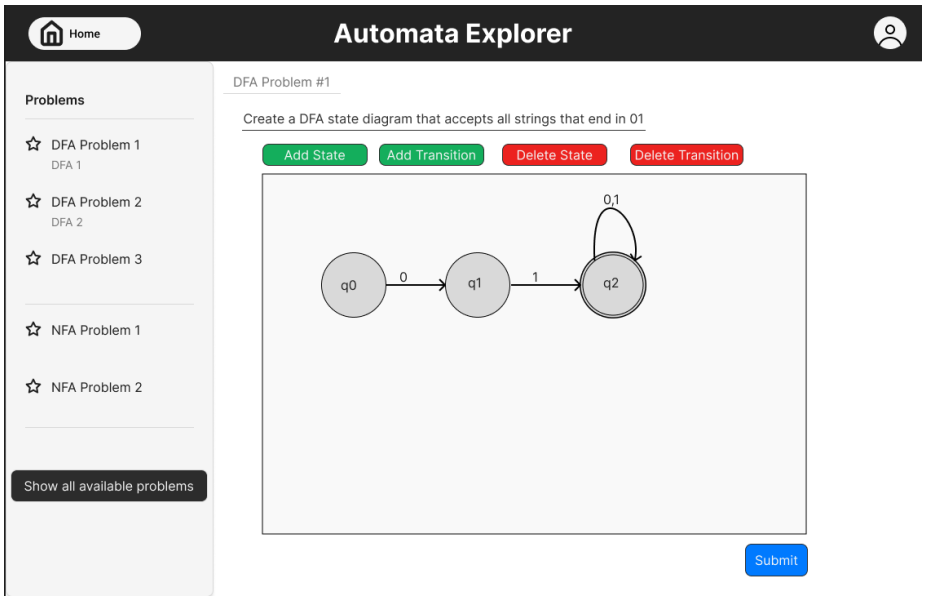

Figure 2: UI/UX Mockup

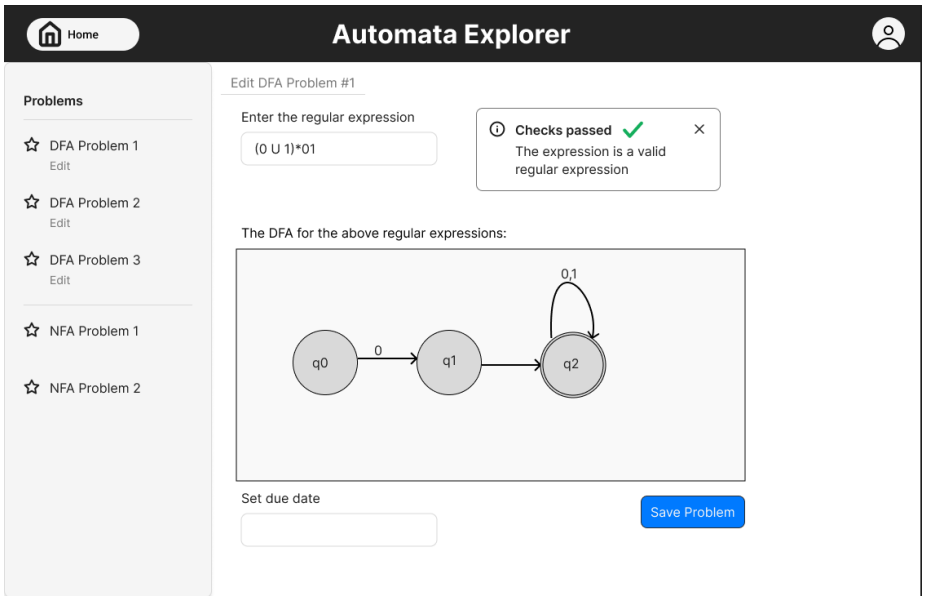

Figure 3: UI/UX Mockup

11 Project Plan

Figure 4 shows the Gantt chart for the project, outlining the tasks and their estimated durations. The Gantt chart also includes the task of recording a video demonstration of the project, and the writing of the dissertation.

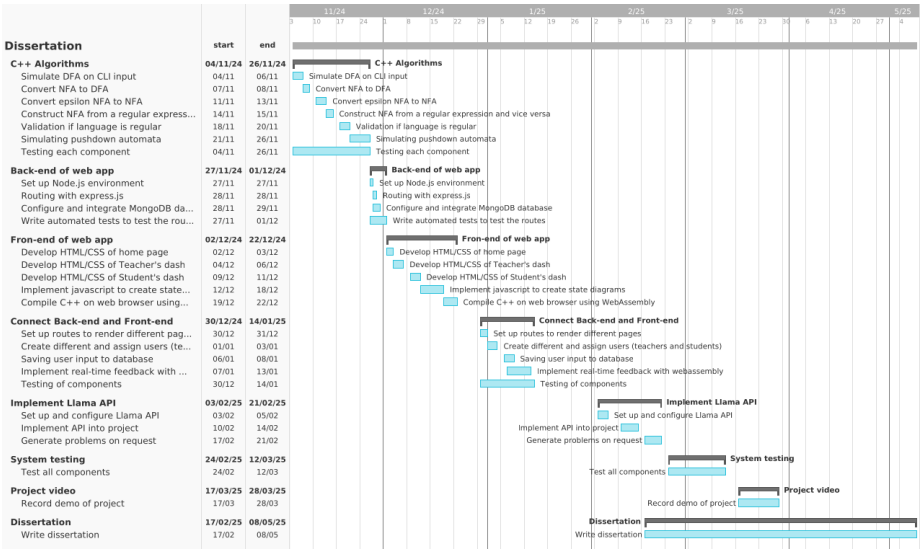

Figure 4: Gantt Chart

12 Risks & Contingency Plans

Risks	Contingencies	Likelihood	Impact
Hardware failure	Regular backups of code (by using GitHub) and data (provided by MongoDB). GitHub will also	Low	Loss of work and project delays
Software failure	Regular testing and debugging	Medium	Project delays
Running out of time	Regularly reviewing project plan and adjusting deadlines	Medium	Unfinished project
Programming problems	Regular testing and debugging	High	Project delays
Llama API downtime	Store AI generated problems in the database which can be accessed if needed.	Low	Reduced functionality of the AI model

13 References

References

[1] M. Sipser, *Introduction to the Theory of Computation*. Cengage Learning, 3rd ed., 2012.

[2] J. Esparza and M. Blondin, *Automata Theory: An Algorithmic Approach*. Cambridge University Press, 2023.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. The MIT Press, 3rd ed., 2009.

[4] MongoDB, "MongoDB Atlas Web Page." `https://www.mongodb.com/cloud/atlas`. Accessed: 2024-10-25.

[5] Mongoose, "Mongoose Web Page." `https://mongoosejs.com/`. Accessed: 2024-10-25.

[6] Llama, "Llama API Web Page." `https://docs.llama-api.com/quickstart`. Accessed: 2024-10-25.

[7] Mozilla Developer, "Mozilla Developer Web Page." `https://developer.mozilla.org/en-US/docs/WebAssembly`. Accessed: 2024-10-25.

[8] Emscripten, "Emscripten Web Page." `https://emscripten.org/`. Accessed: 2024-10-25.

[9] Express, “Express.” https://expressjs.com/. Accessed: 2024-10-25.

[10] Bootstrap, “Bootstrap.” https://getbootstrap.com/. Accessed: 2024-10-25.

[11] GitHub, “GitHub.” https://github.com/. Accessed: 2024-10-25.

[12] bcrypt, “bcrypt.” https://www.npmjs.com/package/bcrypt. Accessed: 2024-10-25.

[13] Figma, “Figma.” https://www.figma.com/. Accessed: 2024-10-25.